

<typename T> и <class T> са еквивалентни

Темплейти - позволява писане на код, независим от типа.

- Генерира се код по време на компилация.
- Типът се извежда по време на компилация.
- Типът T трябва да поддържа всички използвани операции, инак грешка при инстанциране.
- Има темплейтни класове

Темплейтен клас с множествен брой параметри

```
template <typename T, typename U>
```

- 1) T и U са параметри за типове
- 2) При създаване на обект задаваме конкретните типове
- 3) Могат да бъдат различни типове
- 4) Темплейт може да има повече от 1 параметър, тип или константа

Темплейтен клас с дефолтен тип

```
template <typename T=int>
```

```
class Box{
```

```
...
```

```
{;
```

```
Box<> b(42);
```

↓
празно е ⇒ int по подразбиране

• Шаблоните могат да имат параметри различни от типове, например: числа, указатели, bool.

Зависи как ще се използва шаблона, но често са необходими специалните функции (Rule of Five) да са разписани, оператори за сравнение и const коректност.

Темплейтна специализация - позволява различна имплементация на шаблон за конкретен тип или форма на тип.

- Общ шаблон към специален случай.
- Видовете са:
 - първичен шаблон - накрая използва първичния template
 - пълна специализация - търси първо нея
 - частична специализация (само за класове) - след това най-специфичната

Variadic Templates - поддържат неограничен брой параметри
- полезно за универсални функции и логване

typename ... Args - неограничен брой типове

Args... args - параметрите съответстват на всеки тип

Concepts (в C++20) - механизъм за описване на ограничения върху шаблонни параметри. Позволяват да кажем какви свойства трябва да има тип, за да бъде използван в шаблон

- Concepts може да описва: операции, типове членове, ~~време~~ връщани типове, свойства (наследяване и т.н.)

Полиморфизъм чрез concepts

- Позволява compile-time полиморфизъм. Дефинира се интерфейс чрез concept, който проверява наличието на определени методи/свойства на типа.
- Функции или класове, използващи този concept, работят с всякакви типове, отговарящи на условията, без да се използват виртуални функции.
- Компиляторът избира правилните методи още при компилация.
- Постигахме полиморфно поведение без runtime overhead.

STL - част от стандартната библиотека и представлява колекция от готови, темплейтни компоненти за работа с данни и алгоритми.

Цел: Разделя данните от алгоритмите и ги свързва чрез итератори.

Състои се от:

- 1) Контейнери - съхраняват данни - темплейтни класове
- 2) Алгоритми - обработват данните - темплейтни функции, които работят с итератори
- 3) Итератори - свързват контейнерите с алгоритмите - обобщени указатели
- 4) Функционални обекти (functors, lambdas) - поведение, логика